

OpenShift

FOR COMPLETE BEGINNERS

LEARN • DEPLOY • CONFIGURE • SECURE • TROUBLESHOOT
REAL-WORLD CONTAINERS AND APPLICATIONS



**CONTAINERS
AND PODS**



**DEPLOYMENTS
AND SERVICES**



**ROUTES
AND NETWORKING**



**SECURITY
AND RBAC**



**TRUBLESHOOTING
AND REAL LABS**



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OpenShift: What Is It?

1. WHAT IS OPENSIFT?

- OpenShift is a **container application platform** built by **Red Hat**.
- It is based on **Kubernetes** but adds enterprise features, security, developer tools and operational capabilities.
- OpenShift helps you build, deploy, run and manage containerized applications at scale.

2. WHY ORGANIZATIONS USE IT?

- ✓ Provides a complete, production-ready Kubernetes platform.
- ✓ Stronger security and access controls.
- ✓ Built-in developer and DevOps tools.
- ✓ Simplifies application deployment and management.
- ✓ Trusted and supported by Red Hat.
- ✓ Used in enterprises, government and regulated industries.

3. KEY PROBLEMS IT SOLVES

- ✓ Running applications consistently across different environments.
- ✓ Managing containers at scale.
- ✓ Providing secure multi-tenant environments.
- ✓ Simplifying Kubernetes operations.
- ✓ Giving developers self-service tools.
- ✓ Meeting enterprise security and compliance requirements.

4. HOW IT RELATES TO CONTAINERS AND KUBERNETES

- Applications are packaged in **containers** (e.g. Docker containers).
- OpenShift uses Kubernetes to orchestrate, run and manage those containers.
- OpenShift adds additional features such as developer experience, security, networking, monitoring and enterprise operations.

5. SIMPLE RELATIONSHIP DIAGRAM



6. REAL WORLD EXAMPLE



7. JUNIOR ENGINEER INSIGHT

Think of OpenShift as an enterprise-ready Kubernetes platform. You still work with the same container and Kubernetes concepts, but OpenShift adds many features out of the box so you can deploy and manage applications more easily and securely in real-world environments.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Kubernetes vs OpenShift



WHAT IS KUBERNETES?

- Kubernetes is an open-source container orchestration platform.
- It automates the deployment, scaling and management of containerized applications.
- It provides the core building blocks for running containers at scale.
- It is flexible, but you need to integrate and manage many additional tools yourself.



WHAT IS OPENSIFT?

- OpenShift is a Kubernetes-based platform built by **Red Hat**.
- It includes Kubernetes plus additional enterprise features, security defaults and developer tools.
- It provides an integrated, opinionated platform for building and running containerized applications.
- It makes Kubernetes easier, more secure and more production-ready for organizations.

KEY DIFFERENCES

FEATURE	KUBERNETES	OPENSIFT
 Nature	Open-source project (CNCF)	Kubernetes-based platform (built by Red Hat)
 Core Function	Container orchestration	Container orchestration plus enterprise platform capabilities
 Additional Tools	You install and manage many tools separately (e.g. CI/CD, registry, monitoring)	Comes with integrated developer tools, registry, monitoring, logging and more
 Security	Provides basic security features that you must configure	Stronger security defaults, Security Context Constraints (SCC), RBAC and multi-tenancy built-in
 Application Access	Uses Ingress (needs setup)	Provides Routes (simpler external access for applications)
 Ecosystem	Large ecosystem, more DIY	Integrated platform with Operators and tested enterprise solutions
 Target Users	Developers and organizations that want a flexible, lightweight platform	Organizations that want a complete, secure, enterprise-ready Kubernetes platform

KEY TAKEAWAY



- ✓ Kubernetes is the foundation.
- ✓ OpenShift adds the tools, security, developer experience and enterprise features on top of Kubernetes.
- ✓ OpenShift is ideal when you want a complete, production-ready platform with less manual setup.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



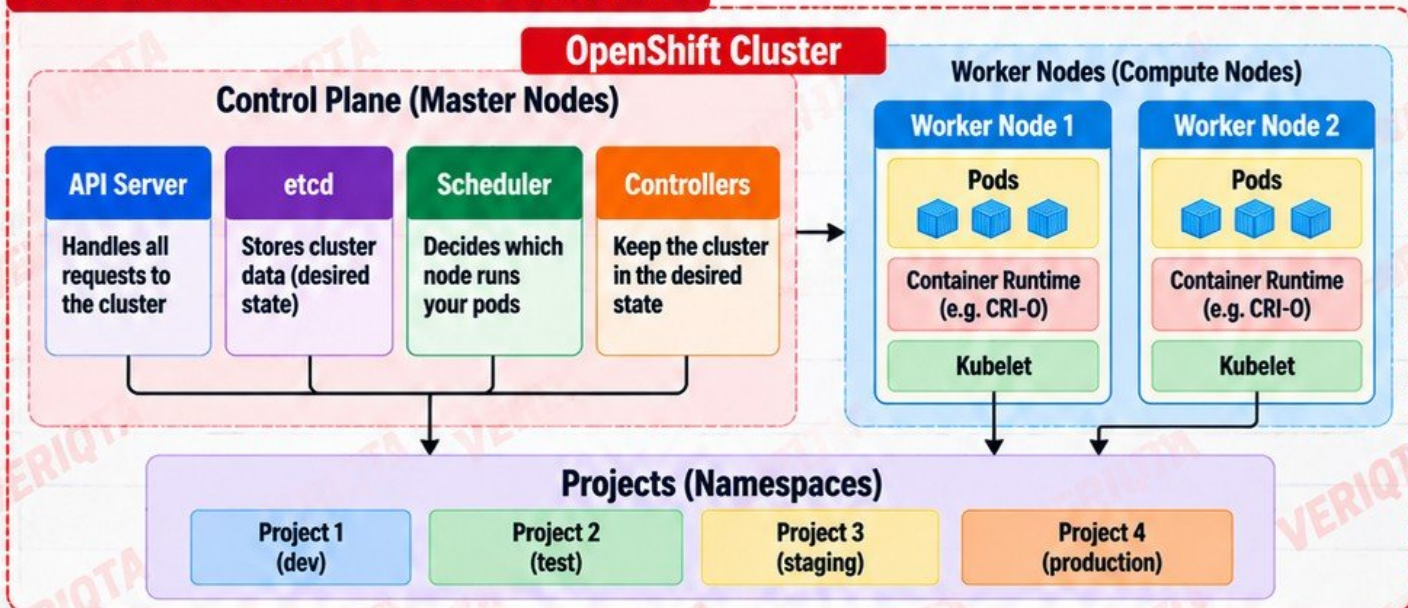
@veriqta

OpenShift Architecture

1. SIMPLE MENTAL MODEL



2. OPENSIFT ARCHITECTURE OVERVIEW



3. KEY COMPONENTS EXPLAINED

	API Server	Entry point for all requests. CLI (oc), web console and other components communicate with the API server.
	etcd	Distributed key-value store that keeps all cluster configuration data.
	Scheduler	Decides which worker node should run a new pod based on resources and rules.
	Controllers	Continuously monitor the cluster and take action to maintain the desired state (e.g. keep pods running).
	Worker Nodes	Run your application containers (pods). Each node has a container runtime and a kubelet.
	Projects	A way to organize and isolate resources. Similar to Kubernetes namespaces.
	Workloads	Your applications running as pods, managed by deployments, statefulsets and other controllers.

4. HOW IT ALL WORKS TOGETHER

- 1 You create a resource (e.g. Deployment) using oc or the web console.
- 2 The request goes to the **API server**.
- 3 The scheduler decides which worker node should run the pod.
- 4 The controller creates and manages the pods.
- 5 The pod runs on a **worker node**.
- 6 Resources are organized inside a **project**.
- 7 OpenShift keeps monitoring and ensuring everything stays in the desired state.

5. IMPORTANT THINGS TO REMEMBER

- ✓ OpenShift is built on Kubernetes.
- ✓ The control plane manages the cluster.
- ✓ Worker nodes run your applications.
- ✓ Projects provide isolation for teams.
- ✓ Controllers keep workloads running.
- ✓ OpenShift adds enterprise features, security defaults and developer tools on top of Kubernetes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

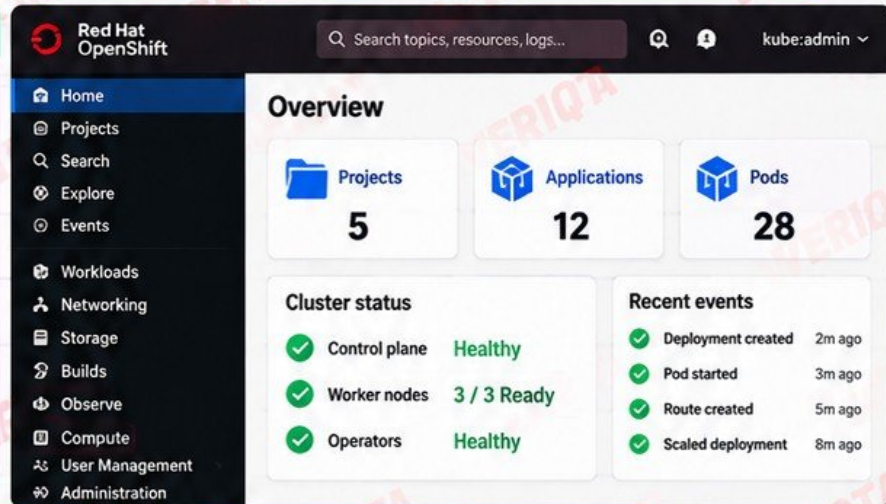


@veriqta

Your First OpenShift Cluster Tour

1. OPENSIFT WEB CONSOLE

- The OpenShift web console provides a graphical interface to manage your cluster.
- You can use two perspectives: Administrator and Developer.
- It helps you manage projects, applications, networking, storage, monitoring and more without using the CLI.



2. ADMINISTRATOR PERSPECTIVE

- Used by cluster administrators.
- Manage the entire cluster.
- Create and manage projects.
- Manage nodes, storage, networking.
- Manage security, users, roles and operators.
- Monitor cluster health and resources.



Common tasks in Administrator view:

- Manage projects and quotas
- View and manage nodes
- Configure storage and networking
- Manage users and roles (RBAC)
- Monitor cluster status
- Manage Operators

3. DEVELOPER PERSPECTIVE

- Used by developers and application teams.
- Focuses on building and deploying applications.
- Easier and simplified interface.
- Work with projects, applications, builds and routes.
- Ideal for day-to-day development tasks.



Common tasks in Developer view:

- Create and manage applications
- Deploy from container images or source code
- View pods, logs and status
- Expose applications using Routes
- Manage environment variables and secrets
- Edit and redeploy applications

4. KEY SECTIONS IN THE CONSOLE



Projects

- Logical separation for teams.
- Create, view and manage projects.



Workloads

- Deployments
- Pods
- ReplicaSets
- Jobs and CronJobs
- View logs and status.



Networking

- Services
- Routes (external access)
- Ingress
- Network policies



Storage

- PersistentVolumes
- PersistentVolume Claims
- StorageClasses



Monitoring

- Metrics, dashboards
- Cluster and application health
- Events and alerts

5. WHERE ENGINEERS PERFORM COMMON TASKS

- Deploy and manage applications → Workloads section
- Expose applications to users → Networking → Routes
- Manage storage for stateful apps → Storage section
- Check application and cluster health → Monitoring section
- Manage users, roles and permissions → Administration
- Install and manage Operators → Administration → Operators

6. JUNIOR ENGINEER INSIGHT



- Use the Developer perspective for day-to-day application work.
- Use the Administrator perspective when you need cluster-level access.
- Explore the console to understand where everything is. This makes you faster when you start working on real projects.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Meet the **oc** CLI

1. WHAT IS oc?

- **oc** is the **OpenShift** command line interface (CLI).
- It is based on **kubectl**, but includes additional OpenShift features and resources.
- You use **oc** to interact with your OpenShift cluster from a terminal.

2. oc vs kubectl



kubectl

- Original Kubernetes CLI
- Works with any Kubernetes cluster
- Manages standard Kubernetes resources



oc

- Built for OpenShift
- Includes OpenShift specific resources (Routes, Projects, Operators, etc.)
- Works with Kubernetes resources too

3. INSTALLATION

The **oc** CLI is usually provided by your organization or can be downloaded from Red Hat.

Verify installation:

```
$ oc version
```

Client Version: 4.x.x

Kubernetes Version: 1.x.x

4. LOGGING IN TO A CLUSTER

Login using the cluster URL provided by your administrator.

```
$ oc login https://api.your-cluster.com:6443
```

You will be prompted for your username and password or use a token.

5. CHECK YOUR IDENTITY

```
$ oc whoami
user1
```

```
$ oc whoami --show-groups
developers
system:authenticated
your-domain-group
```



Shows the current user and the groups you belong to. This helps you understand your permissions.

6. VIEW CLUSTER INFORMATION

```
$ oc cluster-info
```

Shows basic cluster information (API server, console URL, etc.).

```
$ oc get nodes
```

Lists worker nodes in the cluster.

```
$ oc get clusteroperators
```

Shows the status of OpenShift operators.

7. GET HELP

```
$ oc help
```

Shows general help.

```
$ oc help <command>
```

Shows help for a specific command.

```
$ oc explain <resource>
```

Shows the structure and fields of a resource.



Example:

```
$ oc explain pod
```

Shows the details of a Pod resource.

8. EXPLORE COMMON RESOURCES

COMMAND	WHAT IT SHOWS
<code>\$ oc get projects</code>	Lists all projects (namespaces).
<code>\$ oc get pods</code>	Lists all pods in the current project.
<code>\$ oc get deployments</code>	Lists deployments.
<code>\$ oc get services</code>	Lists services.
<code>\$ oc get routes</code>	Lists OpenShift routes (external access).
<code>\$ oc get pv,pvc</code>	Lists persistent volumes and claims.
<code>\$ oc get all</code>	Lists common resources in the current project.

9. SWITCHING PROJECTS

```
$ oc get projects
```

List available projects.

```
$ oc project <project-name>
```

Switch to a project.

10. JUNIOR ENGINEER TIPS

- ✓ Use **oc** instead of **kubectl** when working on OpenShift.
- ✓ Always check which project you are in.
- ✓ Use **oc get** and **oc describe** to understand resources.
- ✓ Use **--help** and **oc explain** when you are unsure.
- ✓ Do not make changes in production without understanding.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Projects and Namespaces

1. WHAT ARE PROJECTS?

- A project in OpenShift is a way to **logically group resources** applications, configurations and users.
- A project is the same as a **Kubernetes namespace**.
- Projects help you **organize** and **isolate workloads** for different teams, environments or applications.

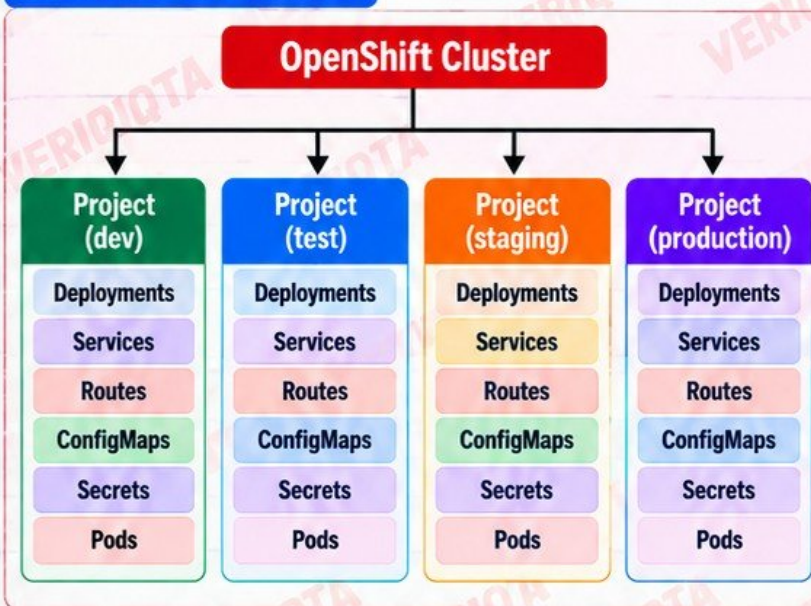
2. WHY LOGICAL SEPARATION?

- ✓ Keeps environments separate (dev, test, staging, production).
- ✓ Improves security and access control.
- ✓ Prevents accidental changes to other applications.
- ✓ Makes resource management easier.
- ✓ Helps teams work independently.
- ✓ Supports quotas and limits.
- ✓ Simplifies monitoring, logging and troubleshooting.

3. PROJECT VS NAMESPACE

Project (OpenShift)	Namespace (Kubernetes)
Term used in OpenShift	Standard Kubernetes term
Same concept	Same concept
Used to group resources	Used to group resources
Supports OpenShift features (quotas, RBAC, etc.)	Core Kubernetes feature

4. VISUAL OVERVIEW



5. COMMON USE CASES

Project	Purpose
dev	Development and initial testing
test	Integration and QA testing
staging	Pre-production validation
production	Live production environment
team-a	Resources for Team A
team-b	Resources for Team B

6. BASIC oc COMMANDS

<code>\$ oc get projects</code>	List all projects.
<code>\$ oc new-project dev</code>	Create a new project.
<code>\$ oc project dev</code>	Switch to a project.
<code>\$ oc get project</code>	See current project.
<code>\$ oc describe project dev</code>	View project details.
<code>\$ oc delete project dev</code>	Delete a project.

7. NAMING GUIDELINES

- ✓ Use lowercase letters, numbers and hyphens.
- ✓ Keep names short and meaningful.
- ✓ Use a consistent naming convention (e.g. dev, test, staging, production).

✗ Avoid:

- Uppercase letters
- Spaces
- Special characters (except hyphen)
- Non-meaningful names like test1234

8. WHY NOT DUMP EVERYTHING IN ONE NAMESPACE?

- ✗ Harder to manage.
- ✗ Security risks (everyone has access).
- ✗ No separation between environments.
- ✗ Accidental changes can affect other applications.
- ✗ Difficult to apply different quotas and policies.
- ✗ Harder to monitor and troubleshoot.

9. JUNIOR ENGINEER TIPS



- ✓ Use a separate project for each environment.
- ✓ Always check which project you are in using `oc project`.
- ✓ Give clear and meaningful project names.
- ✓ Use RBAC to control who can access each project.
- ✓ Organize resources properly from the beginning.



@veriqta



@veriqta



@veriqta



@veriqta



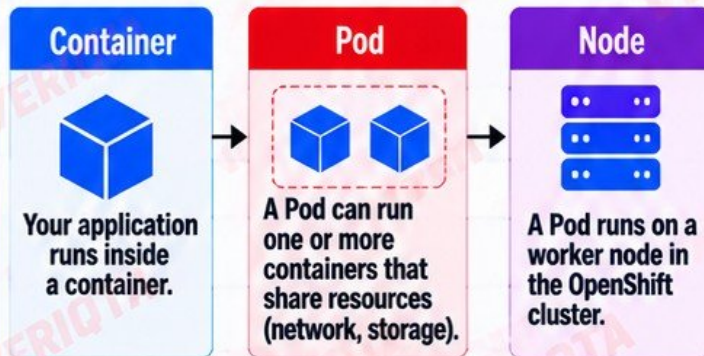
@veriqta



@veriqta

Pods: Where Containers Run

1. CONTAINER → POD → NODE



2. WHAT IS A POD?

- A Pod is the smallest deployable unit in OpenShift.
- A Pod can run one or more containers.
- Containers in the same Pod share the same network (IP address) and can communicate using **localhost**.
- Pods are usually created and managed by higher-level resources like Deployments, StatefulSets, or Jobs.
- **In production, you rarely create Pods manually.**

3. POD STATES

STATUS	MEANING
Running	Pod is running and ready.
Pending	Pod is created but not yet running (e.g. waiting for resources).
Succeeded	Pod completed successfully (usually for Jobs).
Failed	Pod failed to run.
Unknown	Status cannot be determined.
CrashLoopBackOff	Pod keeps crashing and restarting.
ImagePullBackOff	Unable to pull the container image.
Terminating	Pod is being deleted.

5. INSPECTING A POD

```
$ oc get pods
```

NAME	READY	STATUS	RESTARTS	AGE
app-1-5f6d9	1/1	Running	0	2m
db-7d9f4c8	1/1	Running	0	5m

```
$ oc describe pod app-1-5f6d9
```

```
Name:          app-1-5f6d9
Namespace:     dev
Status:        Running
Node:          worker-1/10.0.0.12
IP:            10.128.2.45
Containers:
  app:
    Image:      nginx:1.25
    State:      Running
    Ready:      True
    Restarts:   0
```

4. USEFUL oc COMMANDS

<code>\$ oc get pods</code>	List all pods in current project.
<code>\$ oc get pods -o wide</code>	List pods with more details (node, IP, etc.).
<code>\$ oc describe pod <pod-name></code>	View detailed information about a pod.
<code>\$ oc logs <pod-name></code>	View logs from a pod.
<code>\$ oc logs -f <pod-name></code>	Follow live logs.
<code>\$ oc get pod <pod-name> -o yaml</code>	View full configuration in YAML.
<code>\$ oc delete pod <pod-name></code>	Delete a pod.

6. POD DETAILS YOU SHOULD CHECK

✓ Status	- Is the pod running or failing?
✓ Ready	- Are the containers ready?
✓ Restarts	- Are there frequent restarts?
✓ Node	- Which node is the pod running on?
✓ IP Address	- What is the pod's IP?
✓ Events	- Are there error messages?
✓ Logs	- What is the application logging?
✓ Resource Usage	- Is it using too much CPU or memory?

7. IMPORTANT NOTES



- ✓ Pods are ephemeral (they can be replaced at any time). Do not store important data inside a container.
- ✓ Use Deployments or other controllers to manage pods.
- ✓ Always check logs and events when troubleshooting.
- ✓ A single pod can run multiple containers (sidecar pattern).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Deployments: Keeping Applications Running

1. WHY A SINGLE POD IS NOT ENOUGH?

- A single Pod can fail at any time due to node failure, container crash or maintenance.
- If you run only one Pod, your application will be **unavailable** when it fails.
- Deployments ensure multiple replicas are running, and automatically replace failed Pods.

3. KEY COMPONENTS

Deployment

You manage the Deployment. It defines the desired state (replicas, image, etc.).

ReplicaSet

Created and managed by the Deployment. It ensures the correct number of Pods are running.

Pod

Runs your application containers.

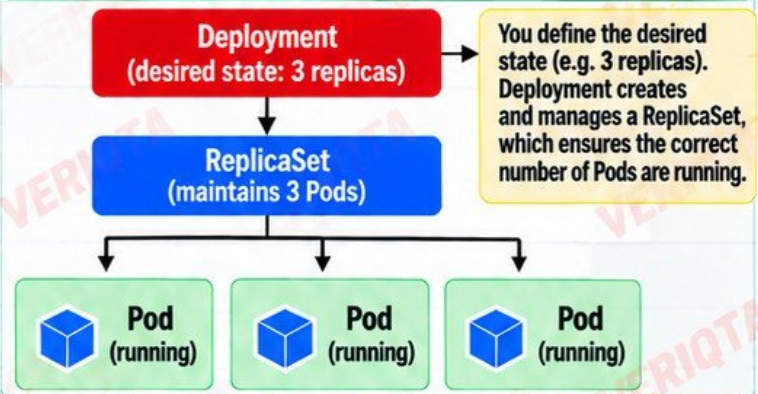
Node

A Pod runs on a worker node in the cluster.

2. WHAT IS A DEPLOYMENT?

- A Deployment manages Pods for you.
- It keeps the desired number of replicas running.
- It uses a **ReplicaSet** to maintain the Pods.
- If a Pod fails, it automatically creates a new one (self-healing).
- It supports **rolling updates** and easy rollbacks.

4. HOW IT WORKS



5. CREATE A DEPLOYMENT (EXAMPLE)

```
$ oc create deployment myapp \
  --image=nginx --replicas=3
```

This creates a Deployment named myapp with 3 replicas using the nginx image.

7. SCALING A DEPLOYMENT

- ✓ You can scale the number of replicas up or down.
- ✓ Scaling helps to handle more traffic or reduce costs.

```
$ oc scale deployment myapp --replicas=5
```



8. ROLLING UPDATES

- A Deployment updates your application without downtime.
- It gradually replaces old Pods with new Pods.
- You can check the rollout status and history, and rollback if needed.

6. USEFUL oc COMMANDS

- | | |
|--|---|
| <code>\$ oc get deployments</code> | List all deployments. |
| <code>\$ oc describe deployment myapp</code> | View deployment details. |
| <code>\$ oc get pods</code> | See the Pods created by the deployment. |
| <code>\$ oc scale deployment myapp \</code>
<code>--replicas=5</code> | Scale the number of replicas. |
| <code>\$ oc rollout status deployment myapp</code> | Check rollout progress. |
| <code>\$ oc rollout history deployment myapp</code> | View rollout history. |
| <code>\$ oc rollout undo deployment myapp</code> | Rollback to previous version. |

9. SELF-HEALING



- If a Pod fails or is deleted, the Deployment automatically creates a new Pod.
- This ensures the desired number of replicas are always running.

10. JUNIOR ENGINEER TIPS



- ✓ Always use a Deployment instead of a single Pod.
- ✓ Set an appropriate number of replicas.
- ✓ Use rollout status to verify updates.
- ✓ Use rollout undo if something goes wrong.
- ✓ Monitor Pods and events if your application is not running.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

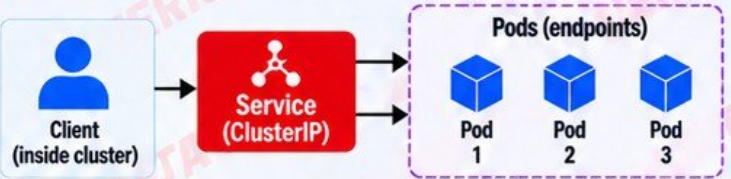
Services: Reaching Your Application

1. WHY POD IP ADDRESSES ARE NOT PERMANENT

- Pods have **temporary IP addresses** that can change when they are deleted and recreated.
- You should not use Pod IPs as application endpoints.
- A Service provides a **stable endpoint** that always points to the correct Pods.
- When Pods are replaced, the Service automatically routes traffic to the new Pods.

2. WHAT IS A SERVICE?

- A Service is a **stable network endpoint** for accessing a group of Pods.
- It uses a **selector** to find the matching Pods.
- It maintains a list of healthy endpoints (Pod IPs).
- It **load balances** traffic to the matching Pods.
- It hides Pod IP changes from users and applications.



3. HOW A SERVICE WORKS

- You create a Service and define a selector (e.g. app: myapp).
- The Service finds all Pods with matching labels.
- The Service creates a list of endpoints (healthy Pod IPs).
- Traffic sent to the Service is load balanced across the endpoints.
- If a Pod is replaced, the Service automatically updates its list of endpoints.

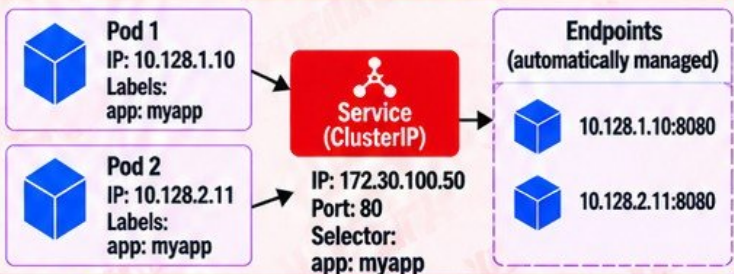
4. SERVICE TYPES (FOCUS: CLUSTERIP)

Type	What It Does	Use Case
ClusterIP	Internal IP accessible within the cluster only.	Default type. Used for internal communication between applications.
NodePort	Exposes the application on each node's IP at a static port.	Testing or simple access (not usually for production).
LoadBalancer	Creates an external load balancer (if supported).	Production access on cloud environments.
ExternalName	Maps to an external DNS name (no proxy).	Accessing external services.

5. KEY CONCEPTS

Selector	Label used to find matching Pods.
Endpoints	List of Pod IPs that the Service routes to.
ClusterIP	Internal virtual IP for the Service.
Port	Port exposed by the Service.
TargetPort	Port on the container inside the Pod.

6. EXAMPLE: SERVICE AND PODS



7. USEFUL oc COMMANDS

<code>\$ oc get svc</code>	List all Services.
<code>\$ oc describe svc <service-name></code>	View Service details.
<code>\$ oc get endpoints <service-name></code>	See the current endpoints (Pod IPs).
<code>\$ oc get pods -l app=myapp</code>	List Pods matching a label.
<code>\$ oc get svc -o wide</code>	View more details including ClusterIP.

8. INTERNAL APPLICATION COMMUNICATION



Applications inside the cluster use the Service name (DNS) to communicate, not Pod IPs. Example:
<http://backend-service:8080>

9. JUNIOR ENGINEER TIPS

- Always use a Service to expose your application internally.
- Do not use Pod IPs as application endpoints.
- Use clear and consistent labels for selectors.
- Check endpoints to make sure your Pods are being discovered.
- Understand the different Service types and when to use them.
- Use meaningful names for your Services.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



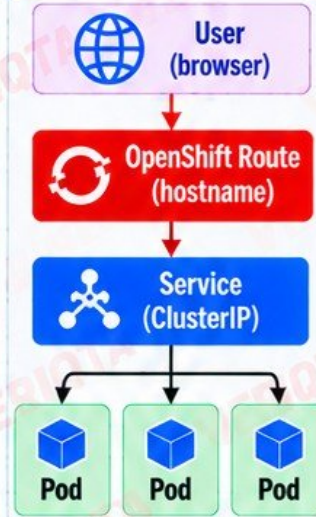
@veriqta

Routes: Exposing Applications

1. WHAT IS AN OPENSIFT ROUTE?

- A Route exposes a Service to external traffic (outside the cluster).
- It provides a **hostname (URL)** that users can access.
- It is an **OpenShift-specific** resource (not part of standard Kubernetes).
- A Route sends traffic to a Service, which then sends traffic to Pods.

2. ROUTE FLOW



3. ROUTE vs SERVICE

	Service (Kubernetes/OpenShift)	Route (OpenShift)
Purpose	Internal communication inside the cluster.	External access to your application.
Type	Standard Kubernetes resource.	OpenShift-specific resource.
Access	ClusterIP, NodePort or LoadBalancer (internal or external).	Provides a hostname (URL) for external users.
Traffic	Routes traffic to Pods using labels/selectors.	Sends traffic to a Service (not directly to Pods).
Example	myapp-svc (ClusterIP)	myapp-route (https://myapp.apps.your-cluster.com)

4. KEY CONCEPTS

Hostname	A DNS name (e.g. myapp.apps.your-cluster.com) that users access in a browser.
TLS	Secures traffic using HTTPS. OpenShift can automatically manage certificates.
Edge Termination	TLS is terminated at the router (common for beginners).
Service	Route sends traffic to a Service.
Router	OpenShift uses a router (usually HAProxy) to handle external traffic.

5. ROUTE TYPES (BEGINNER LEVEL)

Type	What It Does	Use Case
Edge (terminated at router)	TLS is terminated at the OpenShift router. Traffic to the Service is HTTP.	Most common and easiest to use.
Re-encrypt	TLS terminated at router and re-encrypted to Service (HTTPS).	When you want end-to-end encryption inside the cluster.
Passthrough	TLS is not terminated. Traffic is passed directly to the Service.	Advanced use case. Requires the application to handle TLS.

6. CREATE A ROUTE (EXAMPLE)

```
$ oc create route edge myapp-route \
  --service=myapp-svc \
  --hostname=myapp.apps.your-cluster.com
```



This creates an edge Route called myapp-route that exposes the myapp-svc Service using the specified hostname.

8. ACCESS YOUR APPLICATION

- 1 Get the route hostname.
- 2 Open it in a browser (https://...).
- 3 If TLS is enabled, you will see a secure connection (HTTPS).
- 4 You are now accessing your application running inside the cluster.

7. VIEW AND MANAGE ROUTES

\$ oc get routes	List all routes.
\$ oc describe route myapp-route	View route details.
\$ oc get route myapp-route -o yaml	View full configuration.
\$ oc delete route myapp-route	Delete a route.

9. EXAMPLE SCENARIO



10. JUNIOR ENGINEER TIPS

- ✓ Use a Route when you need external access to your application.
- ✓ Do not expose Pods directly.
- ✓ Start with edge termination for simple setups.
- ✓ Always check the generated hostname after creating a Route.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

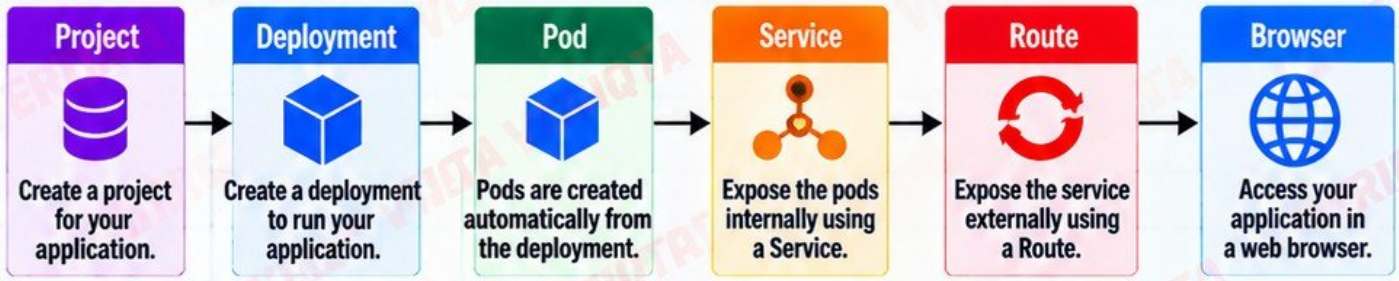


@veriqta

11/20

Deploy Your First Application

1. END-TO-END FLOW



2. CREATE A PROJECT

```
$ oc new-project myapp
$ oc project myapp
```



This creates and switches to a new project called myapp.

3. CREATE A DEPLOYMENT

```
$ oc create deployment nginx \
  --image=nginx \
  --replicas=3
```

- ✓ Creates a deployment named nginx.
- ✓ Uses the nginx image.
- ✓ Starts with 3 replicas (Pods).
- ✓ OpenShift automatically creates a ReplicaSet and Pods.

4. CHECK THE PODS

```
$ oc get pods
$ oc describe pod <pod-name>
```

- ✓ Verify that the Pods are running.
- ✓ Check Pod details, events and logs if needed.
- ✓ Pods are created and managed by the deployment.

5. CREATE A SERVICE

```
$ oc expose deployment nginx \
  --name=nginx-svc \
  --port=80 \
  --target-port=80
```

- ✓ Creates a Service named nginx-svc.
- ✓ Exposes port 80 internally.
- ✓ The Service selects the Pods using labels from the deployment.
- ✓ Other applications in the cluster can now access the application using nginx-svc.

6. CREATE A ROUTE

```
$ oc create route edge nginx-svc \
  --name=nginx-route
```

- ✓ Creates a Route named nginx-route.
- ✓ Exposes the Service to external users.
- ✓ OpenShift assigns a hostname (URL).
- ✓ The route sends traffic to the nginx-svc Service, which then routes to the Pods.



You can also specify a custom hostname and enable TLS if needed.

7. VERIFY THE ROUTE

```
$ oc get route nginx-route
```

- ✓ Check the route hostname (URL).
- ✓ Open the URL in a web browser.
- ✓ You should see the default nginx welcome page.
- ✓ Verify each resource instead of only checking the final webpage.



8. VERIFY EACH RESOURCE

Resource	Command	What to Check
Project	<code>oc get projects</code>	Project exists and you are using it.
Deployment	<code>oc get deployments</code>	Deployment is running with desired replicas.
Pods	<code>oc get pods</code>	Pods are in Running state.
Service	<code>oc get svc</code>	Service has a ClusterIP and correct ports.
Route	<code>oc get route</code>	Route has a hostname and is admitted.
Application	Open in browser	Nginx welcome page is displayed.

9. EXAMPLE OUTPUT

```
$ oc get all
```

```

NAME                                READY   STATUS    RESTARTS   AGE
pod/nginx-5d7c8f9b6-abc12          1/1     Running   0           2m
svc/nginx-svc                      ClusterIP  10.96.123.45  80/TCP     2m
route.route.openshift.io/nginx-route host/    nginx-route-myapp.apps.cluster.com  2m
  
```

10. JUNIOR ENGINEER TIPS



- ✓ Always verify each resource step by step.
- ✓ Understand how traffic flows from Route → Service → Pod.
- ✓ Use meaningful names for your resources.
- ✓ Check logs if the application does not load.
- ✓ Clean up resources when you are done (`oc delete project myapp`).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



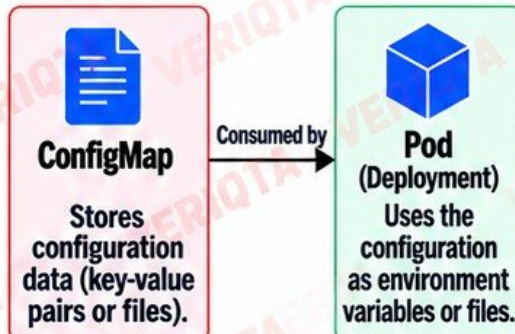
@veriqta

Configuration with ConfigMaps

1. WHY USE CONFIGMAPS?

- Separates configuration from container images.
- Allows you to change configuration without rebuilding the image.
- Makes applications more flexible and easier to manage.
- Keeps the same image across different environments (dev, test, prod).
- Improves maintainability and follows best practices.

2. CONFIGMAP CONCEPT



EXAMPLE USE CASES

- ✓ Application settings (e.g. app.properties)
- ✓ Database connection details
- ✓ API endpoints
- ✓ Feature flags
- ✓ Log levels
- ✓ Non-sensitive configuration data

3. CREATE A CONFIGMAP

From key-value pairs

```
$ oc create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-literal=LOG_LEVEL=info
```

From a file

```
$ oc create configmap app-config \
  --from-file=app.properties
```

4. INSPECT A CONFIGMAP

```
$ oc get configmaps
```

List all ConfigMaps.

```
$ oc describe configmap app-config
```

View details.

```
$ oc get configmap app-config -o yaml
```

View full YAML.

```
$ oc get configmap app-config -o json
```

View in JSON format.

5. CONSUME CONFIGMAP AS ENVIRONMENT VARIABLES

```
env:
- name: APP_ENV
  valueFrom:
    configMapKeyRef:
      name: app-config
      key: APP_ENV
- name: LOG_LEVEL
  valueFrom:
    configMapKeyRef:
      name: app-config
      key: LOG_LEVEL
```

- ✓ Each key in the ConfigMap can be used as an environment variable.
- ✓ The application reads the values at runtime.
- ✓ No image rebuild is required when values change.

6. CONSUME CONFIGMAP AS FILES

```
volumes:
- name: config-volume
  configMap:
    name: app-config

volumeMounts:
- name: config-volume
  mountPath: /etc/app/config
```

- ✓ All keys in the ConfigMap are mounted as files.
- ✓ The application can read the configuration from a directory.
- ✓ Useful for applications that expect configuration files (e.g. app.properties, nginx.conf).

8. CONFIGMAP vs IMAGE REBUILD

Rebuilding the Image for Every Change

- ✗ Slow and unnecessary.
- ✗ Increases build and deployment time.
- ✗ Makes simple configuration changes expensive.
- ✗ Mixes configuration with code.

Using ConfigMaps (Best Practice)

- ✓ Change configuration without rebuilding the image.
- ✓ Keep one image across environments.
- ✓ Easier to manage and maintain.
- ✓ Follows modern DevOps and containerization best practices.

7. UPDATE A CONFIGMAP

```
$ oc edit configmap app-config
```

OR

```
$ oc create configmap app-config \
  --from-literal=APP_ENV=staging \
  --dry-run=client -o yaml |
oc apply -f -
```



After updating a ConfigMap, you may need to restart the Pods (or rollout the Deployment) for the changes to take effect, depending on how your application loads the configuration.

9. JUNIOR ENGINEER TIPS



- ✓ Use ConfigMaps for non-sensitive configuration data.
- ✓ Use meaningful key names.
- ✓ Check how your application reads configuration (environment variables or files).
- ✓ Do not store secrets in ConfigMaps (use Secrets instead).
- ✓ Test the changes and verify the application behavior.
- ✓ Document your configuration for each environment.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



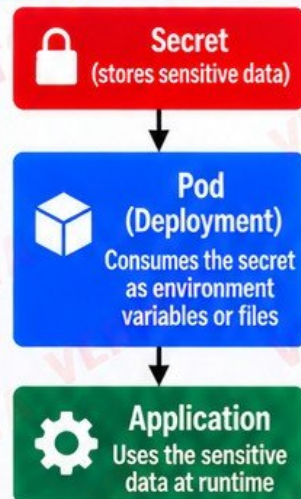
@veriqta

Secrets and Sensitive Configuration

1. WHAT ARE SECRETS?

- Secrets store **sensitive** information such as passwords, API keys, tokens, and certificates.
- They are used by workloads (Pods/Deployments) instead of hardcoding credentials in images or configuration files.
- They help keep sensitive data separate from application code and container images.
- They are OpenShift/Kubernetes resources, similar to ConfigMaps, but designed for sensitive data.

2. SECRET FLOW



3. IMPORTANT POINTS

- ✓ Secrets are stored in etcd (encrypted at rest in OpenShift).
- ✓ They are provided to Pods as environment variables or mounted files.
- ✓ They are not part of the container image.
- ✓ They can be created using key-value pairs or from files.
- ✓ They should only be accessible to authorized users and workloads (RBAC).
- ✓ Always follow secure handling practices for sensitive data.

4. CREATE A SECRET

From key-value pairs

```
$ oc create secret generic db-credentials \
  --from-literal=DB_USER=admin \
  --from-literal=DB_PASSWORD=SecurePass123
```

From a file

```
$ oc create secret generic tls-secret \
  --from-file=tls.crt --from-file=tls.key
```

5. VIEW AND INSPECT A SECRET

```
$ oc get secrets
```

List all Secrets.

```
$ oc describe secret db-credentials
```

View details.

```
$ oc get secret db-credentials -o yaml
```

View full YAML.

```
$ oc get secret db-credentials -o json
```

View in JSON format.

6. CONSUME SECRET AS ENVIRONMENT VARIABLES

env:

```
- name: DB_USER
  valueFrom:
    secretKeyRef:
      name: db-credentials
      key: DB_USER
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-credentials
      key: DB_PASSWORD
```

- ✓ Each key in the Secret can be used as an environment variable.
- ✓ The application reads the values at runtime.
- ✓ Sensitive data is not stored in the image.
- ✓ Do not expose sensitive data in logs.

7. CONSUME SECRET AS A FILE

```
volumes:
- name: secret-volume
  secret:
    secretName: db-credentials

volumeMounts:
- name: secret-volume
  mountPath: /etc/secrets
  readOnly: true
```

- ✓ All keys in the Secret are mounted as files.
- ✓ The application can read the values from a directory.
- ✓ Useful for certificates, keys, or configuration files.
- ✓ Set readOnly: true for better security.

8. WHY BASE64 IS NOT ENCRYPTION

Original value

admin123

Base64 encoded

YWRTaW4xMjM=

- ✗ Base64 is just an encoding method.
- ✗ It can be easily decoded by anyone.
- ✗ It is not encryption and should not be used to protect sensitive data.

9. BEST PRACTICES

- ✓ Do not hardcode credentials in images or source code.
- ✓ Use Secrets for sensitive configuration.
- ✓ Apply RBAC to restrict access to Secrets.
- ✓ Use meaningful secret names.
- ✓ Rotate secrets regularly.
- ✓ Avoid exposing secrets in logs, error messages, or environment listings.
- ✓ Consider using external secret managers (e.g. HashiCorp Vault) for higher security.

10. COMMON USE CASES



Database credentials



API keys and tokens



TLS certificates and private keys



Cloud provider credentials (AWS, Azure, GCP)



Third-party service credentials



Application configuration tokens



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Persistent Storage

1. WHY PERSISTENT STORAGE?

- Container filesystems are **temporary**.
- When a Pod is deleted or rescheduled, any data written inside the container is lost.
- Stateful applications (databases, queues, file storage, etc.) need persistent storage to keep their data.
- Persistent storage allows data to survive Pod restarts, reschedules, and upgrades.
- It separates application data from the lifecycle of individual Pods.

2. KEY COMPONENTS

PersistentVolume (PV)



Cluster resource that provides actual storage (e.g. NFS, EBS, Ceph, etc).

PersistentVolumeClaim (PVC)



Request for storage by a user/example application. It claims a PV based on size, access mode and StorageClass.

StorageClass



Defines how storage is provisioned (type, performance, reclaim policy, etc). Can provision storage dynamically.

Pod



Consumes storage by mounting a PVC as a volume.

3. HOW IT WORKS



StorageClass
(defines how storage is created)



PersistentVolume
(provides storage)



PersistentVolumeClaim
(requests storage)



Pod
(mounts PVC)

4. COMMON OPENSIFT COMMANDS

```
$ oc get pv           # List PersistentVolumes
$ oc get pvc          # List PersistentVolumeClaims
$ oc get sc           # List StorageClasses
$ oc describe pvc <pvc-name> # View PVC details
$ oc describe pv <pv-name>   # View PV details
$ oc describe sc <sc-name>   # View StorageClass details
```

5. EXAMPLE: CREATE A PVC (USING STORAGECLASS)

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: standard
  resources:
    requests:
      storage: 10Gi
```

- ✓ Requests 10Gi of storage.
- ✓ Uses the standard StorageClass.
- ✓ The cluster automatically provisions a matching PV (dynamic provisioning).
- ✓ The PVC can then be mounted by a Pod.

6. MOUNT PVC IN A POD (EXAMPLE)

```
apiVersion: v1
kind: Pod
metadata:
  name: app-with-storage
spec:
  containers:
    - name: app
      image: nginx
      volumeMounts:
        - name: app-data
          mountPath: /usr/share/nginx/html
  volumes:
    - name: app-data
      persistentVolumeClaim:
        claimName: app-data
```

- ✓ Mounts the PVC app-data to /usr/share/nginx/html inside the container.
- ✓ The application can read and write data to this directory.
- ✓ Data remains even if the Pod is deleted or rescheduled.

7. COMMON STORAGE ACCESS MODES

Access Mode	Description
ReadWriteOnce (RWO)	Can be mounted as read-write by a single node (most common).
ReadOnlyMany (ROX)	Can be mounted as read-only by multiple nodes.
ReadWriteMany (RWX)	Can be mounted as read-write by multiple nodes (supported by some storage types).

8. TYPICAL STORAGECLASSES (EXAMPLES)

StorageClass	Use Case	Example Backend
standard	General purpose	OpenShift default (e.g. EBS, Ceph)
fast	High performance	SSD / high IOPS storage
nfs	Shared storage	NFS
thin	Cost efficient	Thin provisioned storage

9. POD AND PERSISTENT DATA RELATIONSHIP



Pod

Reads/Writes data



PVC (claim)



PV (storage)

10. JUNIOR ENGINEER TIPS



- ✓ Use PVCs instead of creating Pods with local storage for important data.
- ✓ Choose the correct StorageClass based on your application needs.
- ✓ Understand the storage access mode required by your application.
- ✓ Check storage usage and capacity regularly.
- ✓ Do not store critical data only in a container's local filesystem.
- ✓ Follow your organization's backup and data retention policies.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Health Checks and Probes

1. WHY HEALTH CHECKS?

- A running container does not always mean the application is healthy.
- The application might be starting, down, or unable to serve requests.
- Probes help Kubernetes/OpenShift detect the real application status.
- They improve reliability, availability, and self-healing.

2. TYPES OF PROBES

Liveness Probe



Checks if the application is still running. If it fails, Kubernetes restarts the container.

Readiness Probe



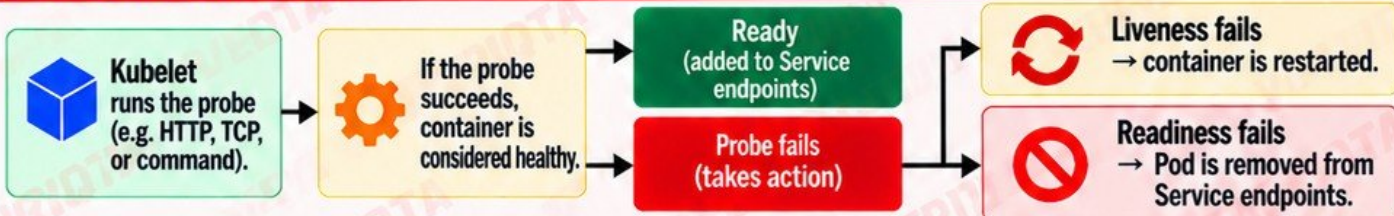
Checks if the application is ready to receive traffic. If it fails, the Pod is removed from Service endpoints.

Startup Probe



Gives the application time to start. Used for slow-starting applications to prevent premature restarts or failures.

3. HOW PROBES WORK



4. PROBE CONFIGURATION (EXAMPLE)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  template:
    spec:
      containers:
        - name: myapp
          image: myapp:latest
          - livenessProbe:
              httpGet:
                path: /health
                port: 8080
              initialDelaySeconds: 30
              periodSeconds: 10
            - readinessProbe:
              httpGet:
                path: /ready
                port: 8080
              initialDelaySeconds: 5
              periodSeconds: 10
            - startupProbe:
              httpGet:
                path: /startup
                port: 8080
              failureThreshold: 30
              periodSeconds: 10
  
```

5. COMMON PROBE TYPES

Type	Description	Example
HTTP Probe	Sends an HTTP request to an endpoint.	httpGet: path: /health port: 8080
TCP Probe	Checks if a TCP port is open.	tcpSocket: port: 8080
Exec Probe	Runs a command inside the container.	exec: command: - cat - /tmp/healthy

6. CHECK PROBE STATUS

\$ oc get pods	Check Pod status.
\$ oc describe pod <pod-name>	View probe events and details.
\$ oc logs <pod-name>	Check application logs.
\$ oc get events	See probe failures and restarts.
 Look for events like: Liveness probe failed, Readiness probe failed, or Container restarted.	

7. WHAT HAPPENS WHEN PROBES ARE WRONG?

✗ Liveness misconfigured	Container may restart repeatedly (crash loop).
✗ Readiness misconfigured	Pod may never receive traffic (application appears down).
✗ Startup misconfigured	Application may be killed before it has time to start (especially slow-starting apps).

8. BEST PRACTICES

- ✓ Use meaningful health endpoints.
- ✓ Set realistic initial delays and timeouts.
- ✓ Use startupProbe for slow-starting apps.
- ✓ Test probe endpoints inside the container.
- ✓ Monitor probe failures in logs and events.
- ✓ Do not rely only on "container running."

9. HEALTH PROBE FLOW (EXAMPLE)



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

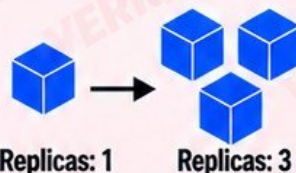
Scaling and Application Updates

1. WHY SCALING AND UPDATES MATTER?

- ✓ Scaling ensures your application can handle more traffic.
- ✓ Updates allow you to deploy new versions without downtime.
- ✓ Rolling updates keep your application available during changes.
- ✓ Rollback helps you recover quickly if something goes wrong.
- ✓ Always verify your application after every production change.

2. SCALING CONCEPT

Scale Replicas



Replicas: 1

Replicas: 3

Scaling increases or decreases the number of Pod replicas to meet demand.

Use Cases

- ✓ Handle increased user traffic.
- ✓ Improve availability and resilience.
- ✓ Distribute load across multiple Pods.
- ✓ Scale down to save resources when demand is low.

3. ROLLING UPDATES CONCEPT

Current Version (v1)



1. Existing Pods are running.

New Version (v2) – Rolling Update



2. New Pods (v2) are created gradually.

Old Pods Terminate (v1)



3. Old Pods (v1) are removed gradually.

Only New Version (v2)



4. Application is updated with zero or minimal downtime.

4. SCALE A DEPLOYMENT

```
$ oc get deployment
# List deployments

$ oc scale deployment nginx --replicas=3
# Scale to 3 replicas

$ oc get pods
# Verify new Pods
```

- ✓ Updates the desired number of replicas.
- ✓ Kubernetes creates or removes Pods.
- ✓ Traffic is automatically balanced across Pods.

5. UPDATE AN APPLICATION

```
$ oc set image deployment/nginx \
  nginx=nginx:1.25
# Update image version

$ oc rollout status deployment/nginx
# Watch the rollout progress
```

- ✓ Changes the container image to a new version.
- ✓ Performs a rolling update by default.
- ✓ Keeps the application available during the update.
- ✓ Verify the rollout status.

6. CHECK ROLLOUT STATUS AND HISTORY

```
$ oc rollout status deployment/nginx
# Check rollout progress

$ oc rollout history deployment/nginx
# View rollout history

$ oc rollout history deployment/nginx --revision=2
# View details of a specific revision
```

- ✓ Shows the current rollout status.
- ✓ Lists previous revisions.
- ✓ Helps with troubleshooting and rollback.

7. ROLLBACK TO A PREVIOUS VERSION

```
$ oc rollout undo deployment/nginx
# Rollback to the previous version

$ oc rollout undo deployment/nginx \
  --to-revision=2
# Rollback to a specific revision
```

- ✓ Reverts to a previous working version.
- ✓ Useful when the new version has issues.
- ✓ Helps restore application quickly.
- ✓ Always verify after rollback.

8. VERIFICATION AFTER UPDATES

- ✓ Check that all Pods are running and healthy.
- ✓ Verify the application is accessible (internal/external).
- ✓ Check application logs for errors.
- ✓ Test key functionality (e.g. login, API, database connection).
- ✓ Monitor for a few minutes to ensure stability.

9. ROLLING UPDATE vs RECREATE

Rolling Update (Recommended)

- ✓ Updates Pods gradually.
- ✓ Keeps the application available.
- ✓ No or minimal downtime.
- ✓ Default strategy.

Recreate Strategy

- ✗ Terminates all old Pods first.
- ✗ Creates new Pods after.
- ✗ Application is unavailable during the update.
- ✗ Used only when downtime is acceptable.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



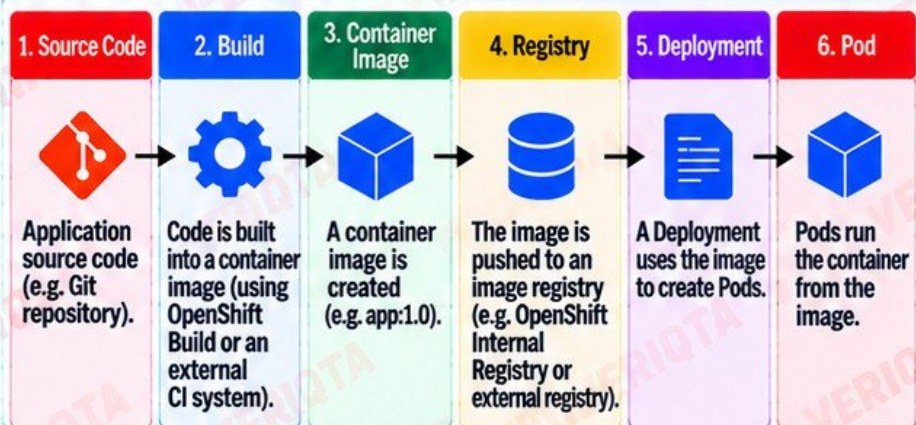
@veriqta

Images, Registries and Builds

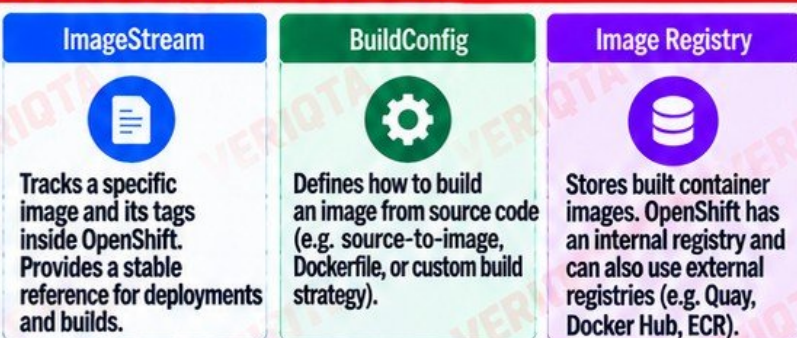
1. WHY CONTAINER IMAGES?

- ✓ Container images package your application and its dependencies in a portable format.
- ✓ The same image runs consistently across different environments.
- ✓ Images make deployments fast, reliable and repeatable.
- ✓ Registries store images so they can be reused and shared.
- ✓ They are the foundation for modern CI/CD and DevOps workflows.

2. END-TO-END FLOW



3. OPENSIFT IMAGE CONCEPTS



4. BUILD STRATEGIES IN OPENSIFT

Source-to-Image (S2I)	Builds an image using a language runtime and your source code.
Dockerfile	Builds an image using a Dockerfile that you provide.
Custom	Uses a custom build process (e.g. Jenkins pipeline or script).
Pipeline	Integrates with OpenShift Pipelines (Tekton) for advanced CI/CD builds.

5. EXAMPLE: CREATE A BUILD AND IMAGESTREAM

1. Create an ImageStream

```
$ oc create imageStream myapp
$ oc get imagestreams
# List ImageStreams
```

2. Create a BuildConfig (using S2I)

```
$ oc new-build nodejs:18-https://github.com/example/myapp.git \
--name=myapp
$ oc start-build myapp
$ oc logs -f build/myapp
# Watch the build progress
```

- ✓ Fetches source code from Git.
- ✓ Builds a container image using a builder image.
- ✓ Stores the built image in an ImageStream.
- ✓ The image can then be used by a Deployment to create Pods.

6. USING IMAGES IN A DEPLOYMENT

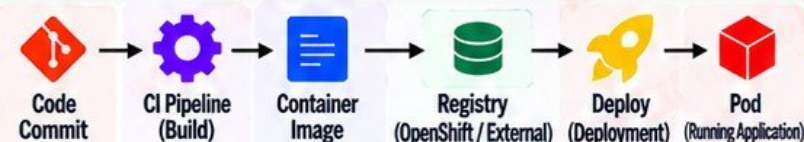
```
$ oc new-app myapp:latest
$ oc get deployments
$ oc get pods
# The deployment uses the image
# from the ImageStream.
```

- ✓ The Deployment automatically pulls the image from the ImageStream or registry.
- ✓ New images can trigger automatic deployments (with triggers).
- ✓ Keeps your application updated with the latest built image.

7. IMAGE REGISTRY OPTIONS

OpenShift Internal Registry	External Registry
	
✓ Built into OpenShift. ✓ Stores images in your cluster. ✓ Accessible via ImageStreams. ✓ Good for development and internal use.	✓ Quay (OpenShift native). ✓ Docker Hub. ✓ AWS ECR / Azure ACR / GCP GAR. ✓ Used for enterprise and multi-cluster environments.

8. HOW THIS FITS INTO CI/CD



9. BEST PRACTICES

- ✓ Use meaningful image tags (e.g. v1.0.0, not latest).
- ✓ Automate builds with CI/CD.
- ✓ Use ImageStreams to decouple deployments from tags.
- ✓ Scan images for vulnerabilities.
- ✓ Use a private registry for sensitive images.
- ✓ Verify deployments after each image update.

OpenShift Security Basics

1. WHY SECURITY MATTERS?

- ✓ Protects your applications, data and cluster resources.
- ✓ Prevents unauthorized access and accidental changes.
- ✓ Helps you follow the principle of least privilege.
- ✓ OpenShift has additional security controls compared to upstream Kubernetes.
- ✓ Containers that run as root locally may behave differently in OpenShift due to security policies.

2. KEY SECURITY COMPONENTS

Users  People or systems that access the cluster.	Service Accounts  Used by Pods and workloads to access the API and other resources.	RBAC  Controls what users and service accounts are allowed to do (roles and bindings).
Roles  Define permissions within a project (namespace).	Role Bindings  Assign roles to users, groups or service accounts.	Security Context Constraints (SCC)  Control what a container is allowed to run as (e.g. user ID, capabilities, volume types).

3. SERVICE ACCOUNTS (EXAMPLE)

```
# Create a service account
oc create sa app-sa

# View service accounts
oc get sa

# Use in a Pod specification
spec:
  serviceAccountName: app-sa
```

- ✓ Service accounts provide an identity for Pods.
- ✓ They can be given specific permissions using RBAC.
- ✓ Each project has a default service account.
- ✓ Use dedicated service accounts for your applications.

4. RBAC: ROLES AND ROLE BINDINGS

```
# Create a role (read-only access to pods)
oc create role pod-reader \
  --verb=get,list,watch \
  --resource=pods

# Bind the role to a user or service account
oc create rolebinding pod-reader-binding \
  --role=pod-reader \
  --serviceaccount=default:app-sa
```

- ✓ Roles define what can be done.
- ✓ Role bindings assign roles to users, groups or service accounts.
- ✓ Use the minimum permissions required (least privilege).

5. SECURITY CONTEXT CONSTRAINTS (SCC)

```
# View available SCCs
oc get scc

# See details of an SCC
oc describe scc restricted
```

- ✓ SCCs control how containers are allowed to run.
- ✓ They can restrict running as root, privileged containers, and certain volume types.
- ✓ This is why a container that runs as root locally may fail in OpenShift if the SCC does not allow it.

6. EXAMPLE: WHY ROOT MAY NOT WORK

Local Docker/Kubernetes



- ✓ Can run containers as root by default.
- ✓ Fewer security restrictions.
- ✓ Useful for local development.

OpenShift



- ✓ Root may be blocked by SCC.
- ✓ Additional security policies are enforced.
- ✓ Designed for multi-tenant environments and stronger security.

7. SECURITY FLOW (SIMPLIFIED)



8. BEST PRACTICES

- ✓ Use dedicated service accounts for applications.
- ✓ Grant only the permissions required.
- ✓ Understand and follow your organization's SCC policies.
- ✓ Avoid running containers as root.
- ✓ Regularly review roles, role bindings and user access.
- ✓ Use namespaces to isolate applications.
- ✓ Follow security guidelines and audit access.

Troubleshooting OpenShift

1. TROUBLESHOOTING MINDSET

- ✓ Use a structured workflow, not guesswork.
- ✓ Look at resource state, events and logs.
- ✓ Identify the real cause, not just the symptom.
- ✓ Verify the fix and monitor the application.
- ✓ Collect evidence before making changes.
- ✓ Understand dependencies (image, storage, network, permissions, configuration).

3. ESSENTIAL OPENSIFT COMMANDS

```
$ oc get pods           # List Pods
$ oc get all            # View all resources
$ oc describe pod <pod-name> # Detailed information
$ oc logs <pod-name>    # View application logs
$ oc get events --sort-by=.lastTimestamp
# View recent events
$ oc describe <resource> # e.g. svc, route, pvc
$ oc get pod <pod> -o wide # More details (node, IP)
$ oc get routes          # List routes
$ oc get pvc,pv          # Check storage
$ oc get sa              # List service accounts
$ oc auth can-i <action> <resource> # Check permissions
```

2. COMMON PROBLEMS AND QUICK CHECKS

Problem	What It Means	First Checks
 Pending Pod	Pod cannot be scheduled.	oc describe pod oc get events
 CrashLoopBackOff	Container keeps restarting.	oc logs oc describe pod oc get events
 ImagePullBackOff	Cannot pull container image.	oc describe pod oc get events
 Failing Route	Application not accessible externally.	oc describe route oc get svc, endpoints Check TLS and DNS
 Bad Configuration	App fails due to wrong config.	oc logs oc describe pod Check ConfigMaps/Secrets
 Failing Probes	Liveness/readiness checks are failing.	oc describe pod oc logs Check probe configuration
 Permission Problem	Not enough permissions (RBAC/SCC).	oc auth can-i oc describe pod Check service account and SCC

4. PENDING POD TROUBLESHOOTING

```
$ oc describe pod <pod-name>
$ oc get events
$ oc get nodes
$ oc get pvc
$ oc get quota
$ oc describe quota
```

- ✓ Check insufficient resources (CPU/Memory).
- ✓ Check unbound PVCs.
- ✓ Check node status and taints.
- ✓ Check quotas and limits.
- ✓ Review events for the exact reason.

5. CrashLoopBackOff TROUBLESHOOTING

```
$ oc logs <pod-name> --previous
$ oc logs <pod-name>
$ oc describe pod <pod-name>
$ oc get events
$ oc get configmap,secret
```

- ✓ Check application logs.
- ✓ Check configuration (ConfigMaps/Secrets).
- ✓ Verify environment variables.
- ✓ Check probes and dependencies.
- ✓ Use --previous to see last logs.

6. IMAGEPULLBACKOFF TROUBLESHOOTING

```
$ oc describe pod <pod-name>
$ oc get events
$ oc get imagestreams
$ oc get secret
$ oc get sa
```

- ✓ Verify image name and tag.
- ✓ Check if image exists in registry.
- ✓ Check image pull secrets.
- ✓ Check network access to registry.
- ✓ Verify service account permissions.

7. FAILING ROUTE TROUBLESHOOTING

```
$ oc describe route <route-name>
$ oc get svc, endpoints
$ oc get pods -l app=<app>
$ oc logs <pod-name>
$ oc get ingresscontroller
```

- ✓ Check route host and TLS.
- ✓ Verify service and endpoints.
- ✓ Confirm Pods are running.
- ✓ Check application logs.
- ✓ Check DNS and firewall (external).

8. FAILING PROBES TROUBLESHOOTING

```
$ oc describe pod <pod-name>
$ oc logs <pod-name>
$ oc get events
```

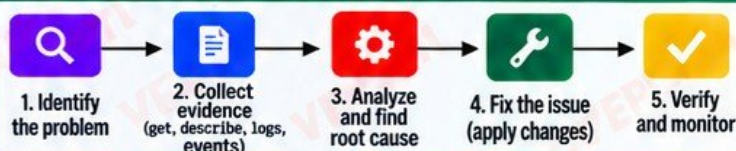
- ✓ Check liveness/readiness settings.
- ✓ Verify application startup time.
- ✓ Confirm probe endpoint/port/path.
- ✓ Check application dependencies.

9. PERMISSION PROBLEMS TROUBLESHOOTING

```
$ oc auth can-i <action> <resource>
$ oc get sa
$ oc describe sa <sa-name>
$ oc describe pod <pod-name>
$ oc get scc
```

- ✓ Check RBAC permissions.
- ✓ Verify service account used.
- ✓ Check Security Context Constraints.
- ✓ Understand what action is blocked.
- ✓ Use least privilege.

10. TROUBLESHOOTING WORKFLOW



11. BEST PRACTICES

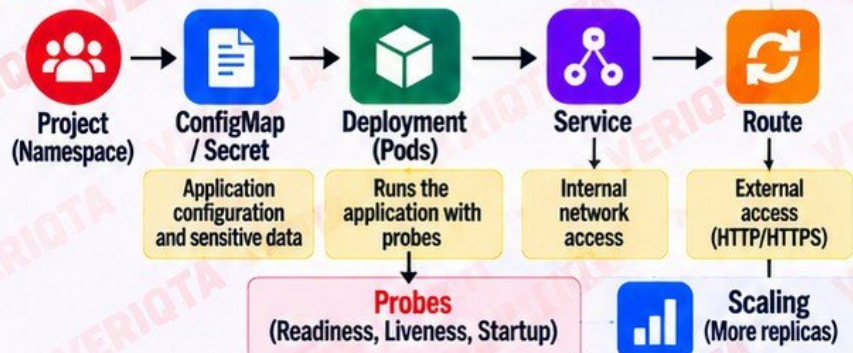
- ✓ Always check events and logs.
- ✓ Use oc describe to get full details.
- ✓ Understand the resource dependencies.
- ✓ Verify application after making changes.
- ✓ Document common issues and solutions.
- ✓ Follow least privilege and security best practices.
- ✓ Do not rely only on "container is running" as a success indicator.

Junior OpenShift Production Lab

1. LAB OBJECTIVE

- ✓ Build a small but realistic application on OpenShift.
- ✓ Use Project → Deployment → ConfigMap/Secret → Service → Route → Probes → Scaling → Logs → Troubleshooting.
- ✓ Understand how the components work together.
- ✓ Deliberately introduce a failure, diagnose it, fix it, and verify recovery.

2. ARCHITECTURE OVERVIEW



3. CREATE A PROJECT

```
$ oc new-project myapp
$ oc project myapp
```

- ✓ Creates a new project (namespace).
- ✓ All resources will be created in this project.

4. CONFIGMAP AND SECRET

ConfigMap (application config)

```
$ oc create configmap app-config \
  --from-literal=APP_COLOR=blue
$ oc get configmap
```

Secret (sensitive data)

```
$ oc create secret generic db-secret \
  --from-literal=DB_PASSWORD=pass123
$ oc get secret
```

5. DEPLOY THE APPLICATION

```
$ oc create deployment myapp \
  --image=nginx:1.25 \
  --replicas=2
$ oc get deployment
$ oc get pods
```

- ✓ Creates a deployment with 2 replicas.
- ✓ Pods will use the configured image.
- ✓ OpenShift automatically manages the pods.

6. EXPOSE WITH SERVICE AND ROUTE

```
$ oc create service clusterip myapp \
  --tcp=8080:80
$ oc expose service myapp \
  --name=myapp \
  --port=8080
$ oc get svc
$ oc get route
```

- ✓ Service provides internal access.
- ✓ Route exposes the application externally.
- ✓ Access using the route URL from a browser.

7. ADD PROBES

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
```

- ✓ Liveness probe restarts the container if it fails.
- ✓ Readiness probe ensures traffic is sent only when ready.
- ✓ Startup probe can be used for slow-starting apps.

8. SCALE THE APPLICATION

```
$ oc scale deployment myapp \
  --replicas=3
$ oc get pods -w
```

- ✓ Increase or decrease the number of replicas.
- ✓ OpenShift automatically handles pod creation and load distribution.

9. VIEW LOGS AND RESOURCE STATE

```
$ oc logs -f deployment/myapp
$ oc get pods
$ oc describe pod <pod-name>
$ oc get events --sort-by=.lastTimestamp
```

- ✓ View application logs.
- ✓ Inspect resource details for more information.
- ✓ Check events for errors and warnings.

10. INTRODUCE A FAILURE

```
# Example: use a wrong image
$ oc set image deployment/myapp \
  myapp=nginx:invalid

# Or set an invalid config
$ oc edit configmap app-config
```

- ✓ The application will fail to start.
- ✓ Observe the pod status (e.g. ImagePullBackOff, CrashLoopBackOff).
- ✓ Use logs, describe and events to find the root cause.

11. DIAGNOSE AND FIX

```
# Check status and events
$ oc get pods
$ oc describe pod <pod-name>
$ oc get events

# Fix the issue (example)
$ oc set image deployment/myapp \
  myapp=nginx:1.25
```

- ✓ Identify the real cause using events and logs.
- ✓ Apply the correct fix.
- ✓ Verify the pods return to the Running state.

12. VERIFY RECOVERY

```
$ oc get pods
$ oc logs deployment/myapp
$ oc get route
```

- ✓ Pods are running and healthy.
- ✓ Application is accessible via the route.
- ✓ Check logs to confirm it is working.

13. LAB COMPLETION CHECKLIST

- ✓ Project created
- ✓ ConfigMap created
- ✓ Secret created
- ✓ Deployment running
- ✓ Service created
- ✓ Route accessible
- ✓ Probes configured
- ✓ Application scaled
- ✓ Failure introduced and diagnosed
- ✓ Issue fixed
- ✓ Application recovered and verified


[@veriqta](https://www.instagram.com/veriqta)

[@veriqta](https://github.com/veriqta)

[@veriqta](https://www.linkedin.com/company/veriqta)

[@veriqta](https://www.telegram.com/veriqta)

[@veriqta](https://medium.com/veriqta)

[@veriqta](https://twitter.com/veriqta)